VISVESVARAYA TECHNOLOGICAL UNIVERSITY

BELAGAVI-590018

Project Phase I (BIC685)

Report on

"SafeDox: A Secure Mobile Application for Document Sharing Using Blockchain, IPFS and End-to-End Encryption "

Submitted in partial fulfillment of the requirements for the VI Sem and award of the degree of

Bachelor of Engineering

in

Computer Science and Engineering

(Internet of Things & Cyber Security Including Block Chain Technology)

Submitted by:

BADAREESH P	**1KS23IC011**
CS JEEVAN SETTY	**1KS23IC014**
NEEHARIKA S	**1KS23IC032**
POORVI P	**1KS23IC039**

Under the Guidance of

Mrs.Poornima H N
Assistant Professor
Dept. of CSE (ICB)

Department of Computer Science and Engineering
(Internet of Things & Cyber Security Including Block Chain Technology)
K S Institute of Technology
No.14, Raghuvanahalli, Kanakapura Main Road, Bengaluru - 560109

2025-26

K S INSTITUTE OF TECHNOLOGY

No.14, Raghuvanahalli, Kanakapura Main Road, Bengaluru - 560109

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

(Internet of Things & Cyber Security Including Block Chain Technology)

CERTIFICATE

Certified that the Project Phase I report entitled **"SafeDox: A Secure Mobile Application for Document Sharing Using Blockchain, IPFS and End-to-End Encryption"** carried out by

BADAREESH P	**1KS23IC011**
CS JEEVAN SETTY	**1KS23IC014**
NEEHARIKA S	**1KS23IC032**
POORVI P	**1KS23IC039**

a bonafide student of **K S Institute of Technology** in partial fulfillment for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering (Internet of Things & Cyber Security Including Block Chain Technology)** of the **Visvesvaraya Technological University, Belagavi** during the year **2026**. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the Report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of Project Phase I work prescribed for the said Degree.

Signature of the Guide	Signature of the HoD	Signature of the Principal
Mrs. Poornima H N	**Dr. Kushal Kumar B N**	**Dr. DILIP KUMAR K**
Assistant Professor	Associate Professor and Head	Principal and Director
Dept. of CSE (ICB)	Dept. of CSE (ICB)	KSIT

K S INSTITUTE OF TECHNOLOGY

No.14, Raghuvanahalli, Kanakapura Main Road, Bengaluru - 560109

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

(Internet of Things & Cyber Security Including Block Chain Technology)

CERTIFICATE

This is to certify that the Project Phase I report entitled **"SafeDox: A Secure Mobile Application for Document Sharing Using Blockchain, IPFS and End-to-End Encryption"** submitted by

BADAREESH P	**1KS23IC011**
CS JEEVAN SETTY	**1KS23IC014**
NEEHARIKA S	**1KS23IC032**
POORVI P	**1KS23IC039**

to the Visvesvaraya Technological University, Belagavi, in partial fulfillment for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering (Internet of Things & Cyber Security Including Block Chain Technology) at KSIT,** is a bonafide record of project work carried out by them under my supervision. The contents of this report, in full or in parts, have not been submitted to any other Institution or University for the award of any degree.

Signature of the Guide

Mrs. Poornima H N
Assistant Professor
Dept. of CSE (ICB)

Signature of the HoD

Dr. Kushal Kumar B N
Associate Professor and Head
Dept. of CSE (ICB)

Declaration

We declare that this project report titled **"SafeDox: A Secure Mobile Application for Document Sharing Using Blockchain, IPFS and End-to-End Encryption"**, submitted in partial fulfillment of the degree of **Bachelor of Engineering in Computer Science & Engineering (ICB) at KSIT.** is a record of original work carried out by us under the supervision of **Mrs. Poornima H N** and has not formed the basis for the award of any other degree, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Name	**USN**	**Signature**
BADAREESH P	1KS23IC011	.....
CS JEEVAN SETTY	1KS23IC014	.....
NEEHARIKA S	1KS23IC032	.....
POORVI P	1KS23IC039	.....

Date:

Place: Bengaluru

Acknowledgement

A number of personalities, in their own capacities have helped us in carrying out this project work. We would like to take this opportunity to thank them all. We would like to express our gratitude to our **MANAGEMENT**, K.S. Institute of Technology, Bengaluru, for providing a very good infrastructure and all the kindness forwarded to us in carrying out this project work in college. We express our sincere gratitude to **Dr. Dilip Kumar K**, Principal and Director, KSIT, Bangalore, for his constant encouragement, support, and motivation in carrying out this work successfully.

We express our sincere gratitude to **Dr. Surekha Borra**, Professor & Dean (R&D), Department of Computer Science & Engineering (ICB), KSIT, Bangalore, for her valuable guidance, continuous encouragement, and insightful suggestions throughout this work. We also extend our heartfelt thanks to **Dr. Kushal Kumar B N**, Associate Professor & Head, Department of Computer Science & Engineering (ICB), KSIT, Bangalore, for his constant support, expert guidance, and valuable suggestions during the course of this work.

We deeply express our sincere gratitude to project coordinator **Mrs. Surekha B** Assistant Professor, Department of CSE (ICB), KSIT, Bangalore for her able guidance, regular source of encouragement and assistance throughout this work. We are thankful to **Mrs. Poornima H N**, Assistant Professor, for being our project guide, under whose able guidance this work has been carried out.

BADAREESH P	1KS23IC011
CS JEEVAN SETTY	1KS23IC014
NEEHARIKA S	1KS23IC032
POORVI P	1KS23IC039

Abstract

Traditional centralized document-sharing systems like Google Drive and Dropbox use service provider-controlled centralized servers in which the encryption keys are held by the service providers, giving users zero control over the shared documents. In this regard, SafeDox uses zero-trust approach, decentralized mobile application that incorporates three different technologies, namely: IPFS through Pinata as a technology of distributed encrypted storage; Ethereum smart contracts as a tool of immutable access control; and hybrid encryption pipeline with AES-256 and ECC encryption algorithms. All shared documents are completely encrypted before being uploaded from the sender's mobile device, and also every file gets its unique SHA-256 Content Identifier (CID) on IPFS which gives it an automatic detection of tampering without need for any centralized infrastructure. The access control in the system is implemented by Solidity smart contracts which are hosted in the Ethereum Sepolia testnet, providing a novel "view-once" function in which access is revoked irreversibly and permanently after reaching the sender-defined view count — a function not provided by any of the 42 papers reviewed between 2018 and 2025. SafeDox consists of four different layers: Flutter/Dart mobile frontend, Python Flask backend, IPFS storage, and Ethereum blockchain.

Contents

List of Figures

List of Tables

Chapter 1

INTRODUCTION

Considering the fact that in modern times, exchanging of documents through digital methods has become an integral part both for official and non-official purposes,there emerges a dire requirement of the provision of a system for exchanging documents that is safe and confidential and ensures full autonomy of the user.Although the currently existing popular systems do offer convenience, yet they lack one major aspect as far as the needs of users are concerned,that of providing the user with absolute sovereignty over their information due to their nature of centralization, thereby making them prone to any sort of threats.This is where SafeDox comes in with a mobile application for exchanging documents with the help of end-to-end encryption.

1.1 Overview

The project addresses a fundamental security gap in mainstream document-sharing platforms such as Google Drive and Dropbox, which depend on centralized servers where service providers retain encryption keys,can access user data at will,and offer no post-share access control [17], [36], [42]. Once a document is shared through such systems,the sender permanently loses control — there is no mechanism to limit the number of views,revoke access, or detect whether the file has been silently tampered with in transit [7], [8], [21].SafeDox was built to eliminate all these vulnerabilities through a zero-trust, decentralized approach that ensures no intermediary server ever possesses plaintext document data at any point in the sharing lifecycle [18], [35].

The system integrates three core technologies to achieve this,first, the IPFS via Pinata stores encrypted documents across a global peer-to-peer network,removing any single point of failure [21], [35].Every file uploaded to IPFS receives a unique SHA-256 Content Identifier (CID) that changes completely if even a single bit of the file is altered, delivering 100% tamper-detection

accuracy without requiring any additional centralized infrastructure [6], [27].Second,Ethereum Smart Contracts written in Solidity and deployed on the Sepolia testnet act as immutable, autonomous access governors [8], [42].When a document is shared, the sender records the CID, the receiver's Ethereum address,and a configurable view limit permanently on-chain;once the view counter reaches zero, access is irrevocably blocked with no override possible by any party,including the developers themselves [7], [25].Third,a hybrid AES-256 and ECC encryption pipeline ensures every document is encrypted entirely on the sender's mobile device before leaving the phone [27], [40]. The AES session key is wrapped using the receiver's Elliptic Curve public key, meaning only the receiver's hardware-stored private key can unlock and decrypt the document [8], [36].

Architecturally,SafeDox is structured across four independent layers:a Flutter/Dart mobile application that performs all cryptographic operations on-device,a lightweight Python Flask backend that coordinates IPFS uploads and blockchain calls while never accessing plaintext, IPFS via Pinata for decentralized encrypted storage, and the Ethereum Sepolia blockchain for trustless and immutable access governance [6], [18], [42]. The project's defining novelty is its "view-once" smart contract mechanism — a feature absent from all 42 research papers surveyed between 2018 and 2025 — in which access is automatically and permanently revoked after a sender-defined number of views, enforced entirely by immutable on-chain logic with no possibility of override [7], [8], [25].Combined with client-side mobile end-to-end encryption,CID-based tamper detection,and right-to-erasure through IPFS unpinning while preserving the on-chain audit trail,SafeDox represents a complete and deployable zero-trust document-sharing system that advances beyond all prior work reviewed in the literature [21], [27], [36], [42].

1.2 Motivation

The rapid expansion of digital communication has made document sharing an indispensable activity across professional and personal domains.Mainstream platforms used such as 'Google Drive','Dropbox' and 'WeTransfer' have simplified file exchange among users,yet they also introduce a fundamental and largely unresolved tension between usability and security.These services rely on centralised cloud architectures, meaning every document a user uploads ultimately resides on servers owned and controlled by a third party.The provider retains the encryption keys,which grants them and,by legal compulsion,government authorities the ability to access private content at any time [42].

Beyond provider side access,centralised storage presents a systemic risk:a single successful breach exposes every user's data simultaneously.The 2012 Dropbox incident,in which 68 million user credentials were compromised,illustrates how attractive a centralised repository is as an attack target

[35].More recently, research has demonstrated that even when files are encrypted at rest,metadata visible on centralised servers can leak sensitive behavioural patterns about users and their sharing habits [42].

A second dimension of the problem concerns access control after sharing.Once a document is transmitted via conventional means-email,a shared link,or a cloud folder-the sender permanently loses the ability to govern its use. The receiver may forward, copy,or redistribute the file without restriction. No existing mainstream platform provides a mechanism to enforce that a document may be opened only a finite number of times,or to revoke access instantaneously and irreversibly once a sharing event has concluded [8], [35].

The introduction of blockchain technology and decentralised storage protocols presents a credible path to resolving these limitations.Ethereum smart contracts enable autonomous,immutable rule enforcement without relying on any central authority [23], [42].The InterPlanetary File System (IPFS) offers content-addressed, distributed storage that eliminates single points of failure and provides inherent tamper detection through cryptographic content identifiers [21], [35].Hybrid encryption combining AES-256 and Elliptic Curve Cryptography (ECC) has been validated as an effective approach for securing documents in mobile environments while maintaining acceptable performance [27], [40].

Despite these advances, a survey of the literature from years 2018 to 2025 reveals that no existing work brings together mobile-first architecture, client-side end-to-end encryption, blockchain-enforced view-count limits, and decentralised IPFS storage in a single deployable system [8], [27], [42]. SafeDox is motivated precisely by this gap — the need for a practical, lightweight, and genuinely trustless document-sharing application that places control firmly in the hands of the sender(user),not the platform(application).

1.3 Objectives

- To develop a secure client-side encryption system using AES-256 and ECC to ensure end-to-end data protection
- To implement decentralized document storage using IPFS with CID-based integrity verification
- To design blockchain-based access control using Ethereum smart contracts for secure sharing
- To enable controlled document access with user-based permissions and view-count restrictions

1.4 Organization of Report

Chapter 1-Introduction: Establishes the foundation of the work.The chapter opens with an overview of the current document-sharing landscape and the security shortcomings of centralized platforms.The motivation for pursuing a decentralized, blockchain-driven alternative is then developed, followed by a precise statement of the four technical objectives that govern the project.The chapter closes with this organizational outline.

Chapter 2-Literature Survey: Conducts a structured survey of forty-two research papers spanning the period 2018 to 2025.he reviewed works collectively cover blockchain-based access control, IPFS-integrated storage,hybrid encryption schemes, and secure file-sharing architectures across domains including healthcare, vehicular networks, and federated learning.Each paper is examined for its methodology,key contributions, and limitations.A comparative table consolidates the findings,after which recurring gaps in the existing literature are identified and a formal problem statement is derived.

Chapter 3-Requirement Engineering: Addresses requirement engineering.The hardware and software tools selected for the project are catalogued and justified.Conceptual models are then presented in the form of use case and sequence diagrams.The chapter concludes with a Software Requirements Specification covering both functional requirements(which define what the system must do) and non-functional requirements(which govern performance, security and usability constraints).

Chapter 4-Project Planning: Deals with project planning and scheduling.A Gantt chart maps each development phase to its corresponding timeline, making dependencies between tasks and milestone deadlines explicit.

Chapter 5-System Design: Presents the proposed system design.The four-layer decentralized architecture is described first, followed by the end-to-end methodology,a decomposition of the system into its constituent modules,interface design, data structure design, and the algorithmic logic underpinning the encryption pipeline and smart contract access-control mechanism. Since implementation has not yet commenced,all content in this chapter reflects the planned design.

Chapter 6-Conclusion and Future Scope: Draws conclusions from the work completed in Phase 1 and identifies directions for future development, including potential extensions such as quantum-resistant cryptography, biometric authentication, and multi-chain deployment.

Chapter 2

LITERATURE SURVEY

The literature survey is organized into four phases corresponding to the key technical layers of the proposed system: Encryption Layer,Decentralized Storage Layer, Blockchain Governance Layer,and Mobile Integration and End-to-End Encryption Layer.Each phase presents a focused review of relevant research works,a comparative summary table,and an analysis of the limitations identified in the existing literature.

2.1 Encryption Layer

The Encryption Layer is the basic security layer of the proposed system because it provides security in terms of confidentiality,integrity,and access control for sensitive documents and records.Centralized encryption systems have faced various types of problems such as key management,insider threats,and lack of scalability. Recently,there have been several efforts to combine blockchain technology with cryptographic techniques like Attribute-Based Encryption (ABE),Proxy Re-Encryption (PRE), Zero-Knowledge Proof(ZKP),and threshold cryptography.

In this stage, emphasis is placed on the ability of current encryption techniques to enhance secure data sharing within decentralized networks,healthcare,Internet-of-Things (IoT), and blockchain-based systems.From the analysis of the selected papers, it can be seen that encryption integrated with blockchain can greatly enhance trust,privacy protection,and access control.

Table 2.1: Literature Summary Table — Encryption Layer

Sl. No	Paper / Authors / Year	Methodology	Findings	Limitations
1	Jabri et al. [1], 2025	Combined blockchain with proxy re-encryption for securing medical IoT records	Improved secure sharing of medical records with reduced unauthorized access	Increased computational complexity
2	Ullah et al. [5], 2025	Fine-grained attribute-based encryption with decentralized storage	Enhanced privacy and access control in EHR management	High encryption overhead
3	Berrios Moya et al. [11], 2025	Blockchain academic verification using zero-knowledge proofs	Improved credential verification privacy	ZKP computation cost is high
4	Cai et al. [16], 2024	Attribute-based encryption with outsourced decryption and blockchain	Reduced decryption burden on users	Dependence on third-party outsourcing
5	Zhang et al. [20], 2024	Blockchain with zero-knowledge proof transaction security	Strengthened privacy preservation in distributed systems	Increased latency during verification
6	Yuan et al. [22], 2024	Blockchain-based intellectual property authentication	Improved ownership verification and privacy	Scalability issues in large deployments
7	Tariq et al. [37], 2022	Self-configurable ransomware prevention for IoMT	Enhanced protection against ransomware attacks	Limited evaluation environments
8	Chen et al. [40], 2021	Threshold proxy re-encryption for secure IoT data sharing	Secure decentralized sharing without exposing original keys	Complex key management mechanisms

From the above studies, there is a clear emphasis on the need for incorporating encryption algorithms into the blockchain system to provide for secure and private data sharing. The Attribute-Based Encryption algorithm provides access control while the Proxy Re-Encryption algorithm provides for secure delegation of the data without disclosing the encryption key. Zero-Knowledge Proof algorithm provides for enhanced privacy.

Despite the enhanced level of security and privacy that blockchain systems employing encryption have provided, there still exist many obstacles.For instance, most of these systems require a lot of computational power due to the involved use of cryptography,like ABE and ZKP. Key management is also very challenging in decentralized environments.There can be dependency problems in the outsourcing and proxy systems.

2.2 Decentralized Storage Layer

The Decentralized Storage Layer concentrates on providing secure, distributed,and immutable storage of files and documents through the use of technologies like InterPlanetary File System (IPFS),which is coupled with blockchain technology.The conventional centralized cloud storage solutions are characterized by single points of failure,lack of transparency,risks of data manipulation, and dependence on third parties.In order to overcome the mentioned problems, modern research suggests decentralized systems wherein blockchain guarantees integrity and access control, whereas IPFS offers distributed storage functionality.

In this stage,research papers dealing with the application of blockchain and IPFS for secure file sharing,academic credentials validation,document management,healthcare record storage,and decentralized repositories will be considered.

Table 2.2: Literature Summary Table — Decentralized Storage Layer

Sl. No	**Paper / Authors / Year**	**Methodology**	**Findings**	**Limitations**
1	Haque et al. [4], 2025	Blockchain and IPFS-based source code repository hosting	Improved decentralized repository security and integrity	Middleman approach introduces dependency
2	Kumar et al. [6], 2025	Blockchain document verification using IPFS	Enhanced document authenticity and tamper resistance	Limited scalability testing
3	N and Yadlapalli [7], 2025	Blockchain-based secure document sharing	Improved secure sharing and transparency	High transaction cost
4	Patil et al. [8], 2025	Blockchain and IPFS with smart contract access control	Fine-grained access control for file sharing	Smart contract execution delays
5	Belfqih and Abdellaoui [10], 2025	Blockchain authentication with IPFS-based IoT data management	Improved decentralized IoT data handling	Increased computational overhead

Sl. No	Paper / Authors / Year	Methodology	Findings	Limitations
6	Tiwari et al. [17], 2024	Hybrid blockchain-based file sharing system	Enhanced reliability and data availability	Hybrid architecture complexity
7	Kadam et al. [18], 2024	Secure file storage using blockchain technology	Improved integrity and decentralized security	Storage redundancy overhead
8	Agrawal et al. [19], 2024	Web3.0 document security using blockchain	Improved decentralized document protection	Limited real-world implementation
9	Bin Saif et al. [21], 2024	Secure distributed storage using IPFS and blockchain	Efficient decentralized storage and retrieval	Retrieval latency issues
10	Sangeeta and Nam [24], 2023	IPFS-blockchain vehicular data storage with keyword search	Secure searchable distributed storage	Search efficiency limitations
11	Jadhav et al. [27], 2023	Decentralized document storage using IPFS	Enhanced document security and availability	Limited large-scale evaluation
12	Rahman et al. [28], 2023	Blockchain and IPFS-based credential verification	Improved credential transparency and trust	Smart contract dependency
13	Sultana et al. [32], 2023	IPFS-blockchain framework for certificate fraud reduction	Reduced certificate forgery risks	Scalability concerns
14	Athanere and Thakur [34], 2022	Semi-decentralized IPFS data sharing architecture	Improved secure and efficient data sharing	Partial centralization remains

Sl. No	Paper / Authors / Year	Methodology	Findings	Limitations
15	Huang et al. [35], 2022	Secure file sharing using IPFS and blockchain	Enhanced confidentiality and integrity	IPFS availability dependency
16	Anwar et al. [36], 2022	AnonChain secure file sharing framework	Improved anonymity and secure sharing	Increased communication overhead
17	Pincheira et al. [38], 2022	Smart contract and distributed storage-based dataset sharing	Trusted decentralized dataset exchange	Complex governance mechanisms
18	Mani et al. [41], 2021	Hyperledger Healthchain with IPFS health record storage	Patient-centric secure healthcare storage	Healthcare interoperability challenges
19	Steichen et al. [42], 2018	Blockchain-based decentralized IPFS access control	Improved decentralized authorization management	Limited scalability analysis

As is evident from the reviewed literature,the combination of the blockchain technology and IPFS allows achieving a viable decentralized storage system which can be used to ensure integrity, transparency,and fault tolerance.The use of smart contracts allows managing access control issues, whereas the implementation of distributed storage minimizes the reliance on centralized servers.Such solutions have been extensively used in the areas of document verification,healthcare,certificates,IoT, and repository management.Overall,the reviewed research reveals that decentralized storage technologies contribute to increasing the availability and integrity of information,as well as to providing secure peer-to-peer information sharing.

However,some problems that have not yet been solved persist despite the benefits of using decentralized storage systems.Many of these systems lack scalability,latency in retrieval,and redundant storage.The cost of executing smart contracts and other fees associated with blockchain transactions might hinder their efficiency.Others have some centralized elements making it difficult for them to be fully decentralized.

2.3 Blockchain Governance Layer

Blockchain Governance Layer aims at developing trust,transparency,authentication, and decentralized decision-making in blockchain enabled technologies.The governance layer makes sure that all the aspects of data transactions,access control, authentication, and operations of organizations can be handled in a secured manner without depending upon any centralized authority.Smart contracts and consensus of blockchains act as important elements in validating the process and enforcing security policies.

Table 2.3: Literature Summary Table — Blockchain Governance Layer

Sl. No	Paper / Authors / Year	Methodology	Findings	Limitations
1	Nguyen et al. [2], 2025	Blockchain-enabled trustworthy data sharing framework	Improved secure and transparent data sharing	Complex integration with existing systems
2	Wu et al. [12], 2025	Blockchain-based secure data transaction and privacy preservation scheme	Enhanced transaction integrity and privacy	High computational overhead
3	Cardenas-Quispe and Pacheco [13], 2025	Blockchain-based degree verification prototype	Improved academic integrity and certificate authenticity	Limited deployment scalability
4	Mathwale and Ramisetty [25], 2023	Blockchain-based inter-organizational secure file sharing	Secure collaboration between organizations	Dependence on blockchain network performance
5	Farabi et al. [29], 2023	Blockchain-powered academic credential verification system	Reduced credential forgery and verification time	Regional implementation limitations
6	Hammad et al. [30], 2023	Blockchain-based decentralized software version control architecture	Improved transparency and traceability in version control	Increased storage overhead
7	Khor et al. [33], 2023	Lightweight blockchain-based supply chain ownership management protocol	Efficient interoperable ownership management	Limited interoperability evaluation

The aforementioned studies reveal how essential the role of blockchain governance is in creating a reliable,secure, and decentralized management system.The automation of verification and access control through smart contracts makes the system independent of intermediaries and promotes trust among the involved parties.There is practical application of blockchain governance in verification, supply chain management,software versioning, and secure collaborations within organizations.All these systems prove the usefulness of blockchain governance in increasing accountability, traceability, and data integrity.

Despite the advantages of blockchain governance mechanisms, there are still several limitations. Scalability and interoperability are some of the issues faced by most blockchains when used on large scales.There is also latency involved in smart contracts as well as the consensus mechanism.Another issue associated with the use of blockchain governance is the significant amount of storage required.

2.4 Mobile Integration and End-to-End Encryption

The theme for the Mobile Integration and End-to-End Encryption Layer involves enabling the use of technologies for secure communication, real-time accessibility, and secure information exchange within mobile and distributed platforms.In light of the increase in the number of IoT devices,mobile healthcare systems, and federated learning applications,there is a need for secure integration methods to safeguard sensitive information being transferred.This chapter analyzes research papers where blockchain, federated learning,IoT security,mobile healthcare integration,and end-to-end encryption have been used.

Table 2.4: Literature Summary Table — Mobile Integration and End-to-End Encryption

Sl. No	**Paper / Authors / Year**	**Methodology**	**Findings**	**Limitations**
1	Rangwala et al. [3], 2025	Blockchain-enabled federated learning framework	Improved decentralized learning security and trust	High communication overhead
2	Javed et al. [9], 2025	AI-driven blockchain and federated learning for EHR sharing	Enhanced secure healthcare data sharing	Complex AI integration
3	Shahzad et al. [14], 2025	Zero-trust medical image sharing using blockchain and IPFS	Improved secure decentralized medical image access	High storage and transmission cost
4	Sameera et al. [15], 2024	Privacy-preserving blockchain-based federated learning systems	Improved user privacy in distributed learning	Increased training latency
5	Goh et al. [23], 2023	Blockchain-enabled federated learning reference architecture	Verified secure and scalable federated learning implementation	Resource-intensive deployment
6	Sonkamble et al. [26], 2023	Secure EHR transmission using blockchain technology	Improved integrity and confidentiality of health records	Blockchain scalability issues

Sl. No	Paper / Authors / Year	Methodology	Findings	Limitations
7	AlAsqah and Moulahi [31], 2023	Federated learning and blockchain integration for IoT privacy protection	Enhanced IoT privacy and decentralized learning	Complex synchronization mechanisms
8	Zhao et al. [39], 2021	Privacy-preserving blockchain-based federated learning for IoT devices	Secure distributed learning for IoT environments	Limited performance in heterogeneous networks

It is quite clear from the review of the literature that the use of blockchain technology along with federated learning and end-to-end encryption creates an increase in the security of the communication and the decentralization of the intelligence system.With the implementation of such technologies, privacy can be ensured and reliance on central data can be minimized,allowing for the creation of a safe mobile and IoT environment.Federated learning allows for training models with encrypted user data.The reviewed literature reveals high applicability of the discussed methods in healthcare, IoT, mobile apps, and distributed AI environments.

However,despite their benefits, there are also some practical issues with the use of mobile integration and end-to-end encryption.The federated learning approaches tend to have high communication and synchronization overhead.The use of blockchain technology can lead to higher latencies and greater resource requirements,particularly in case of mobile and IoT applications with low computational power. Moreover,scalability, interoperability,and real-time processing continue to pose great challenges.

2.5 Gap Analysis

- **Gap 1 – No End-to-End Encryption in Blockchain-IPFS Document Sharing Applications:** The existing blockchain-IPFS applications only concentrate on access control within the permission layer but save the documents on IPFS without encrypting them.
- **Gap 2 – No Enforcement of Post-Sharing Access Control:** The existing schemes provide permanent access to the documents once the sender shares them, and there is no way to revoke access automatically after viewing the document for a certain number of times.
- **Gap 3 – Lack of Mobile First Approach for Decentralized Documents Sharing:** The vast majority of solutions for decentralized documents sharing using blockchain and IPFS are developed as desktop web-based applications or enterprise level server applications.
- **Gap 4 – Computational Complexity of Advanced Cryptographic Systems on Mobile Devices:** Solutions that rely on complex cryptographic schemes such as CP-ABE, PR-E and ZKP provide solid security guarantees, but they are computationally too expensive for mobile devices.

2.6 Problem Statement

The current document sharing services like Google Drive, Dropbox,and email transfers use centralized cloud infrastructure,which is inherently flawed with several security concerns.The most important of these include having a single point of failure,wherein any breach of the server will result in all documents being compromised, service providers gaining unauthorized access to the documents,and the lack of any mechanism to limit the use of shared documents.Moreover,the integrity of the files cannot be guaranteed because there is no method to ensure that the files have not been tampered with.

SafeDox aims to solve these problems using Zero Trust architecture with decentralized implementation,including end-to-end encryption of the documents using AES-256 and Elliptic Curve Cryptography,distributed file storage using IPFS and Cryptographic Content ID for integrity checks,and view count access restriction using Ethereum Smart Contracts.

Chapter 3

REQUIREMENT ENGINEERING

Requirements Engineering uniquely delivers value to the product development process.Engineers can see requirements as they are authored,including all attributes.Requirements are consistently traced to product information,and changes can be consistently managed.Requirements can be reused within projects and across projects.In addition,Requirements Engineering enables you to take a tangible value added step towards a comprehensive systems engineering approach.Requirements are ubiquitous and increasingly complex.To deliver high quality products,requirements need to be captured,communicated,and managed across many dimensions. Requirements are the foundation of a product development process.They are both shared upstream with customers,those that are receiving the end results of those requirements and downstream with the different engineering disciplines.

3.1 Hardware and Software Tools Used

Hardware Tools

- System: Pentium IV 2.4 GHz.
- Storage: 4 GB or higher free hard drive space.
- RAM: 4 GB or higher.
- Any desktop / Laptop system with above configuration or higher level.

Software Tools

Programming Languages

- **Python 3.10+:** The backend runs on Python because Fernet, eciespy, and Web3.py are all available,well-maintained, and work together cleanly in one environment.We looked at Django and dropped it brings far more than a simple API coordinator needs [8], [27].
- **Dart:** Used with Flutter; compiles straight to native ARM on Android, which keeps encryption and decryption fast on phones with limited processing headroom.
- **Solidity ^0.8.0:** The go-to language for Ethereum contracts and the only one with proper production maturity.The 0.8 series fixed integer overflow by default, closing a bug class that caused real losses in older contracts.Vyper did not make the cut the community and documentation are both too thin [30], [42].

Mobile Frontend

- **Flutter SDK 3.x:** One codebase covers Android and iOS, and hot reload keeps iteration quick.React Native was the main alternative we looked at and dropped it because the crypto library situation is not where it needs to be for something security-focused.
- **flutter_secure_storage:** This is where the ECC private key lives inside the Android Keystore hardware-backed keychain.It never hits plain storage and never leaves the device [5], [40].
- **web3dart:** Handles all smart contract interaction from inside the app itself,no external wallet needed.
- **file_picker:** Pulls up the device's own file browser when the user needs to pick a document to share.
- **http**: Takes care of the REST calls going back and forth between Flutter and Flask.

Backend Server

- **Flask 3.x:** Just HTTP routing, nothing more.The backend does not need a database layer or a templating engine it moves ciphertext around, and Flask is the right size for that job.
- **cryptography** (Fernet): AES-256-CBC with HMAC-SHA256 baked in.NIST-approved,runs fast with hardware acceleration, and any attempt to tamper with the ciphertext blows up immediately at decryption [14], [40].
- **eciespy**: Generates ECC-256 key pairs and handles ECIES wrapping of the AES session key. ECC-256 hits the same security bar as RSA-3072 but is much cheaper to run on a phone [40].
- **Web3.py 6.x:** Signs transactions and talks to the smart contract for all three operations — sharing, accessing, and revoking.
- **requests**: Posts the ciphertext to Pinata's API; straightforward HTTP and nothing more is needed here.

Blockchain Toolchain

- **Ethereum Sepolia Testnet:** Behaves exactly like Ethereum mainnet but test ETH from faucets covers all gas costs, so evaluation is free. Hyperledger Fabric came up during planning and was dropped needing a membership service provider puts a central authority right back into the picture [42].
- **Remix IDE:** Writes, compiles, and deploys the smart contract entirely in the browser. No local toolchain setup, which suited the project well.
- **MetaMask / Sepolia Faucet:** Holds the deployment wallet and pulls in the test ETH needed for contract deployment and ongoing transaction fees.

Storage and Cryptography Stack

- **IPFS Protocol:** Files are addressed by a hash of their own content, so the CID doubles as a tamper-detection fingerprint change one bit anywhere and the CID is completely different. AWS S3 was not considered seriously; it is centralised and defeats the point [21], [35].
- **Pinata API:** Keeps the uploaded files pinned and reachable on IPFS without us having to run our own node.The free tier's 1 GB is plenty for academic-scale testing.

- **AES-256 via Fernet:** 2^{256} key space, CBC mode to break up data patterns, and the HMAC tag catches any ciphertext interference before decryption gets anywhere.
- **ECC via ECIES:** Wraps the AES session key asymmetrically; ECC-256 gives RSA-3072-level security with keys small enough that mobile hardware handles the operation without a second thought [40].
- **SHA-256 via IPFS CID:** Same file, same hash, every time. One bit changed anywhere and the hash is unrecognisable — that property is what makes CID-based tamper detection reliable [35].

Development and Version Control

- **VS Code:** Used for both Flutter and Python development across the team.
- **Git / GitHub:** Keeps the codebase under version control and lets all four team members work without stepping on each other.
- **SQLite (optional):** Can cache the document list locally in the Flutter app to make the UI feel snappier between sessions. It holds nothing security-sensitive and is never used to make access decisions — the chain handles all of that.

3.2 Conceptual/Analysis Modelling

The purpose of a conceptual data model is to show as many rules about the meaning and interrelationships among data as are possible. Conceptual data Modeling is typically done in parallel with other requirement analysis and structuring steps during system analysis.

A Conceptual Model is a representation of a system that uses concepts and ideas to form said representation.Conceptual modelling is used across many fields, ranging from the sciences to socioeconomics to software development. Analysis Model is a technical representation of the system. It acts as a link between system description and design model.In Analysis Modelling, information, behavior and functions of the system is defined and translated into the architecture,component and interface level design in the design modelling.

3.2.1 Use Case Diagram

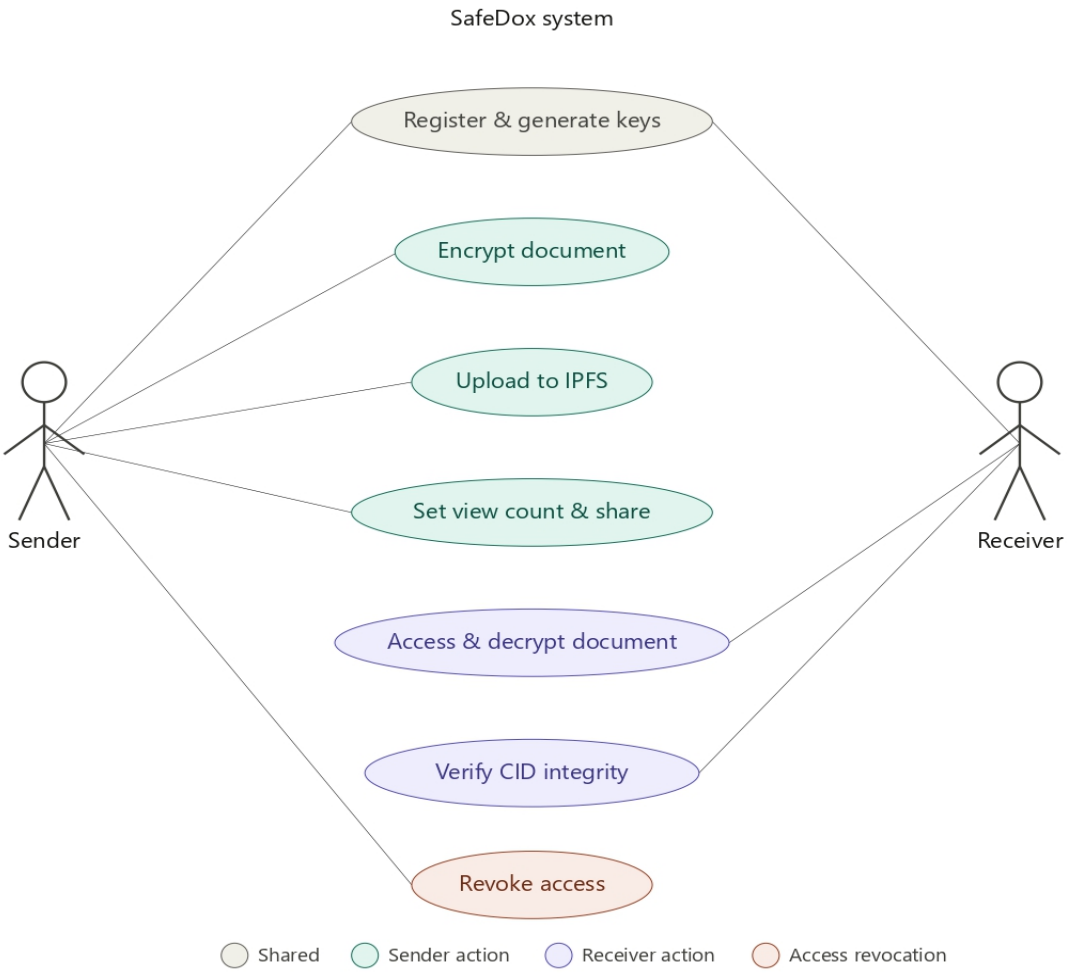

Figure 3.2.1: Use Case Diagram

The diagram covers the two primary actors in SafeDox the sender and the receiver and the actions each can perform within the system.Both actors begin with registering and generating their ECC key pair,which is the one step they share.From there the roles split:the sender encrypts the document on-device, uploads the ciphertext to IPFS,and sets a view count before sharing.The receiver's side involves requesting access,which triggers a CID integrity check against the on-chain record before any decryption is allowed.The sender also holds the exclusive ability to revoke access at any point,cutting off the receiver regardless of how many views remain.

3.2.2 Sequence Diagram

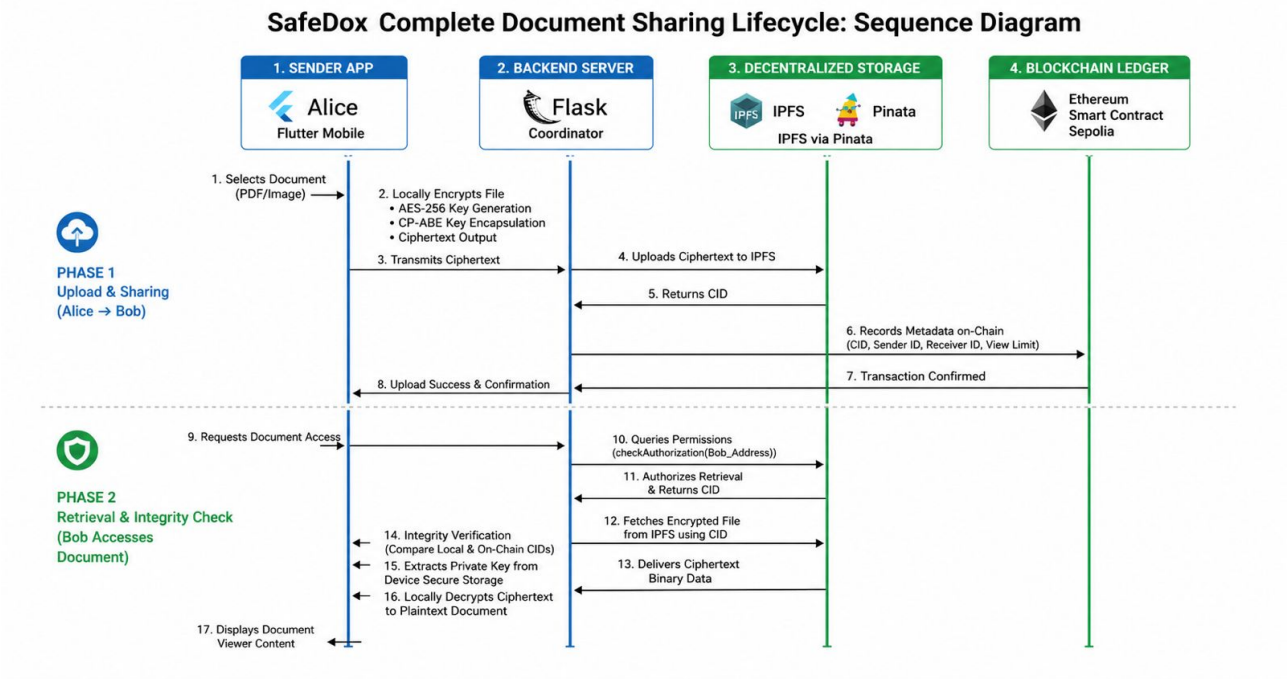

Figure 3.2.2: Sequence diagram.

The sequence diagram traces a full sharing cycle between Alice (sender) and Bob (receiver) across the four system components the Flutter mobile app, the Flask backend, IPFS via Pinata, and the Ethereum smart contract on Sepolia.

The first phase covers upload and sharing. Alice selects a document, and the app encrypts it locally using 'AES-256' before anything leaves the device.The ciphertext is sent to Flask, which uploads it to IPFS through Pinata. IPFS distributes the file and returns a unique CID. Flask then writes the CID, sender ID, receiver ID, and view limit to the smart contract in a single on-chain transaction. Once the ledger confirms the entry, Alice receives an upload success notification.

The second phase covers retrieval.Bob requests access, and the app queries the smart contract with his Ethereum address.The contract runs three checks it validates the receiver identity, confirms the view count is above zero, and decrements the counter before returning the CID.Flask fetches the encrypted file from IPFS using that CID and delivers the ciphertext to Bob's device.The app then recomputes the CID locally and compares it against the on-chain value to verify the file has not been tampered with. If the check passes,Bob's ECC private key is retrieved from the device hardware keystore,the AES session key is unwrapped, and the document is decrypted entirely on-device before opening in the viewer [27], [42].

3.2.3 Functional Requirements

1. **User Registration and Authorisation**

   The solution needs to support user registration and authorisation.The user is supposed to be identified before any actions to work with documents are performed.

2. **Document Upload with Client-Side Encryption**

   AES-256 (Fernet) encryption will be used to encrypt documents on a client-side before sending it to the server.The transmission of AES keys will be done by means of ECC ensuring the confidentiality of the exchange.

3. **Decentralized Storage of Documents with Use of IPFS Protocol**

   IPFS nodes will be used for hosting all the documents encrypted previously.The system will generate CIDs (content identifiers) for each document.

4. **Document Access Rules Based on Smart Contracts**

   The document access control policies will be managed with the use of Ethereum-based smart contracts (Sepolia testnet).Each document will have a smart contract containing recipient information.

5. **Document Distribution**

   The user who shares the document should have an option to specify the list of recipients of the shared document and also to indicate the maximum number of views possible.The number of accesses to the document will be monitored, and after exceeding the allowed view number,the access rights will be withdrawn.

6. **Retrieval & Decryption of Document**

   The recipient with authority should be able to access the document from IPFS by utilizing CID.The system would decrypt the document only when the identity of the recipient and view count are validated by the smart contract.

7. **Validation of Document Integrity Using CID Check**

   When retrieving any document from IPFS,the system must validate the CID.In case there is a mismatch, the document might be tampered with and hence will not be accessible.

8. **Automated Revocation of Access**

   After exhausting the view-count limit for any particular document, the system must automatically revoke further access for that recipient without requiring any manual steps.

9. **REST API Interface for Mobile App & Flask Backend**

   Flutter mobile app will interface with the Flask-based backend through REST APIs.

3.2.4 Non-Functional Requirements

1. **Security**

   All encryption needs to be performed only client-side (E2EE). Neither backend nor IPFS node will ever have access to plaintext files.AES-256 offers symmetric confidentiality protection,whereas ECC is used for asymmetric key exchange security.

2. **Integrity**

   All saved documents have to be verified via CID hash (SHA-256).If there is even one bit difference between two documents,they should have different CID, providing 100% integrity check for all saved files.

3. **Immutability & Trustless operation**

   Logic behind Smart Contract needs to remain immutable and cannot be altered once it was deployed to the Ethereum blockchain (even by developers).Thus, access control rules will be autonomous and immutable.

4. **Efficiency & Lightweight Operation**

   Encryption process used (AES-256 + ECC) needs to be lightweight and performant enough to run on mobile hardware. Any complex encryption algorithm such as CP-ABE, ZKP and PR will be explicitly excluded because of their heavy computation load for mobile hardware.

5. **Scalability**

   IPFS makes sure that the system will be inherently scalable because no centralized server is involved for the storage of documents.The Flask backend and Pinata API need to be able to work concurrently.

6. **Accessibility and Decentralization**

   From the perspective of accessibility and decentralization, it can be noted that since the IPFS is based on peer-to-peer protocols and a distributed network,it would not have any single point of failure and, hence,the documents will remain accessible even in situations where some of the nodes fail to function.

7. **Cross-platforming**

   Considering the fact that the mobile app is going to be developed using Dart and Flutter,its development is supposed to be possible using the same code for both Android and iOS operating systems. Android version 8.0 (API level 26) will be the minimal version to be supported by the application.

8. **Maintainability**

   In order to ease the process of maintenance of the system, modularity needs to be considered by designing the front end, backend, smart contracts and cryptographic functionalities separately.

9. **Usability**

   The system should provide for users to easily upload and share documents without requiring them to know anything about blockchain,IPFS,and cryptography.

10. **Auditability**

    Considering the fact that all transaction regarding document sharing,permission granting and access control are recorded on the Ethereum blockchain network,auditing such transactions becomes easy.

Chapter 4

PROJECT PLANNING

SafeDox's design will be done within a well-defined twelve-week period starting from February to May 2026, which is divided into eight critical steps that include research topic identification,literature review of blockchain access control,IPFS, and hybrid encryption, finding gaps in previous research like no end-to-end encryption,post-sharing access control,and mobile suitability for advanced cryptographic techniques, formulating a problem statement based on vulnerabilities in centralized platforms and proposing an architecture of four layers which include client-side AES-256 and ECC encryption, decentralized IPFS with CID integrity and Ethereum smart contract-based view-count access control.

4.1 Gantt Chart

Gantt charts are project management tools which help in scheduling and planning of any type of project.However, Gantt charts are especially useful in decomposing complex projects into simpler components.It helps in case when tasks must be accomplished on time for managing a large number of stakeholders in the project, especially when there are changes in the scope of the project.A Gantt chart is a tool which allows visualizing the project schedule where each task is depicted using a horizontal bar in a timeline format which makes it easy to identify the interdependencies between the tasks and accomplishing them on time. In case of SafeDox project,a Gantt chart will serve as the road map depicting the phases of the development of the project from reviewing existing literature,analyzing the existing problems and developing the system within twelve weeks.Figure 4.1.1 depicts the Gantt chart for the project.

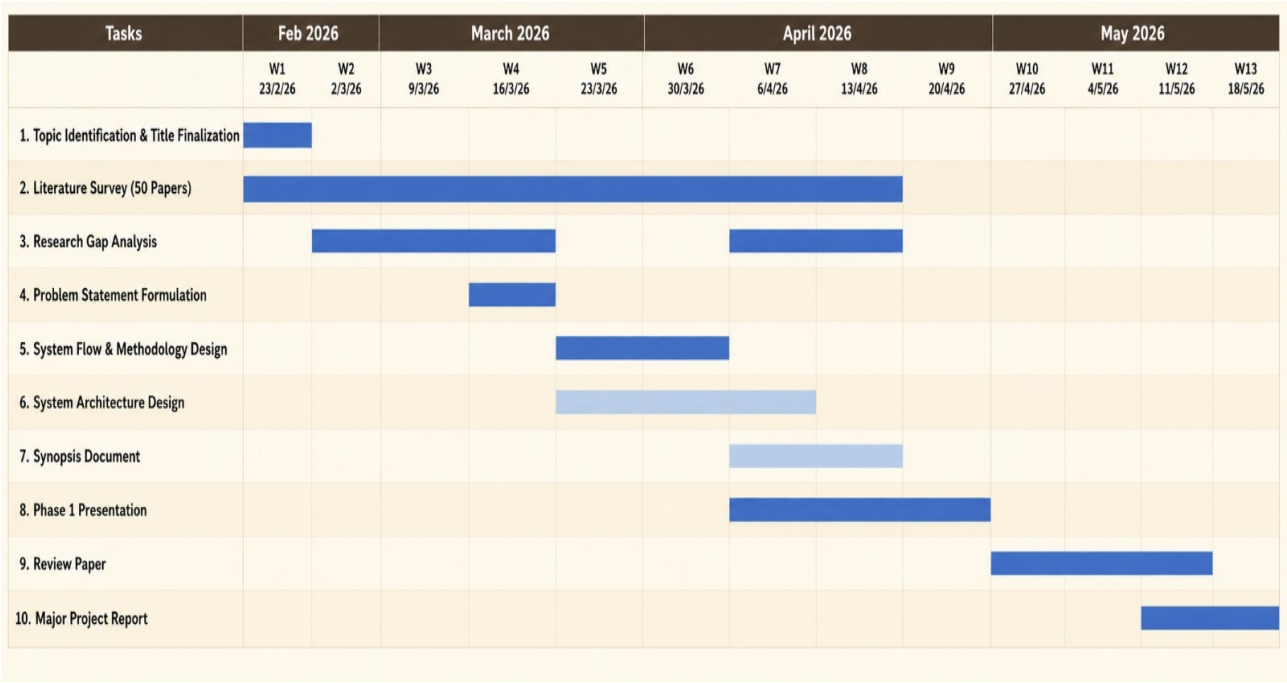

Figure 4.1.1: Gantt Chart

Chapter 5

SYSTEM DESIGN

This section discusses the architecture and algorithms of SafeDox.In this subsection,the internal architecture of the SafeDox application will be analyzed,as well as how the different elements interact and the procedures involved in encrypting the data to ensure security and access control.The decisions made during the design process were influenced by the non-functional requirements that were highlighted in the previous section.

5.1 System Architecture

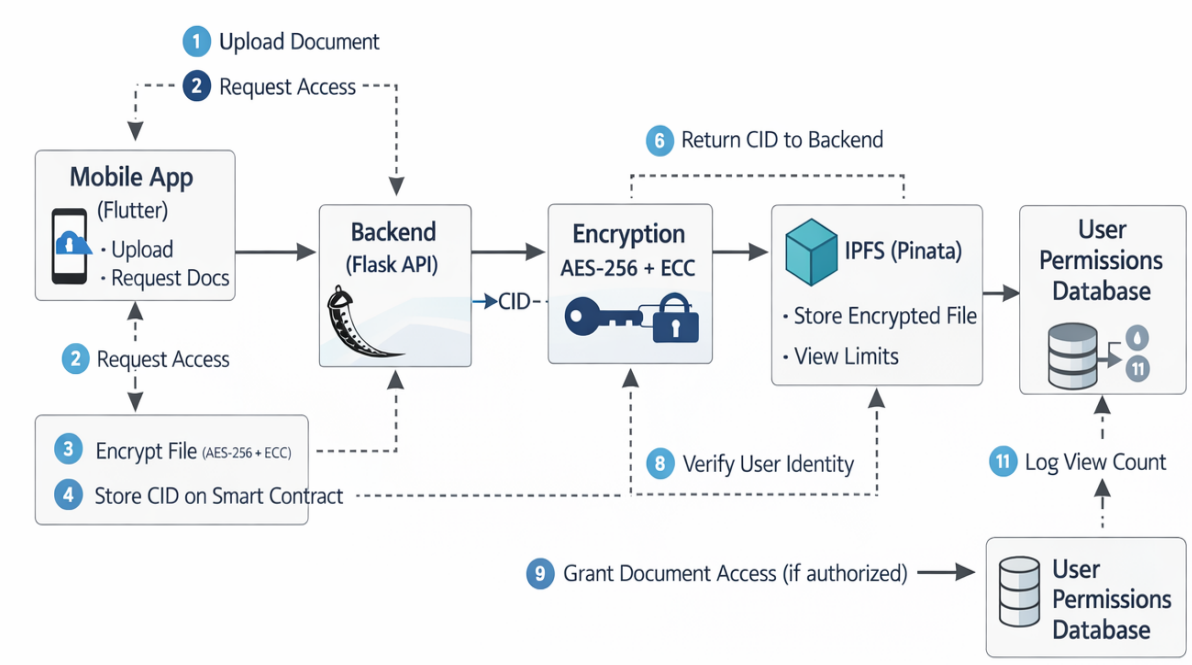

Figure 5.1.1: System Architecture

The SafeDox system starts from the Mobile App (Flutter) which is the main interface for the sender and receiver.The process starts when a sender wishes to send a document,he/she uploads the document through the app (Step 1).The mobile app offers two main actions:document upload and document request .It directly communicates with the Flask API backend to start the workflow. Similarly,when a receiver wishes to access a shared document,he/she sends a Request Access signal (Step 2) back through the same mobile interface,which forwards the request down to the backend for processing.

Backend (Flask API) is the main coordinator of the whole process. As soon as the document is uploaded,it is passed directly to the encryption process (Step 3),where it is encrypted by the hybrid AES-256 and ECC pipeline before being stored anywhere.After encryption, the Content Identifier (CID) is generated by the backend and saved in Ethereum Smart Contract (Step 4),thus providing permanent storing of the cryptographic fingerprint of the document along with its access rules.After that,the encrypted document is sent to IPFS with the help of Pinata and stored together with the view limits rules.Finally,IPFS sends back the CID to the backend (Step 6).

However,whenever a receiver attempts to gain access to a document,a series of checks are done to ensure that the user is granted access.Firstly,the backend does a verification check for the Ethereum identity of the user against the on-chain smart contracts (Step 8).Should the address match the authorized receiver and the view limit not exhausted, the backend will grant access to the document (Step 9) through checking the User Permissions Database that contains the active document sharing permissions.After the access has been granted,every document viewing is recorded (Step 11) in the User Permissions Database,thereby reducing the view limit by one in the smart contracts.When the view limit gets exhausted, the access becomes automatic and permanent.

It is specifically engineered to make sure that each part does not have full control of any document at any stage. The Mobile App never uses plaintext documents,only encrypting the information.The Flask Backend is just a controller,which does not store document content once it transfers it to IPFS.Pinata,as a part of IPFS,keeps only the encrypted document,which can be used for nothing without the receiver's private ECC key.The Ethereum Smart Contract contains only CID and access information but not the document itself,and the User Permissions Database keeps only views statistics and authorization records but not document content.That is,the intentional design of distributing responsibility between five components guarantees that an attacker will not be able to decrypt any document by breaching one layer,like backend or database leak or IPFS access.In order to decrypt a document,the hacker should have access to an encrypted file,receiver's hardware key,and correct authorization,which makes SafeDox safe from external breaches and any insider threat from any layer of the architecture.

5.2 Module Decomposition

SafeDox is split into six modules.Each module owns one job, exposes a defined interface to whatever calls it,and has no visibility into the internal workings of the others.The boundary holds across all three host environments the Flutter app,the Flask backend,and the Solidity contract [17], [25].

Module 1 - Key Management Module

This module owns everything to do with ECC keys.On a fresh install it generates a new ECC-256 key pair on the device,drops the private key into `flutter_secure_storage`,and sends the public key to Flask for registry storage.On later launches it simply confirms the pair exists.When decryption is needed elsewhere in the app,this module is called with a wrapped key as input and returns the recovered AES session key no other module ever touches the private key directly,and the key is cleared from memory the moment the calling operation finishes [5], [40].

Module 2 - Encryption Module

This module lives in Flask and handles the hybrid encryption step.It takes a plaintext byte array and the receiver's ECC public key,generates a one-time AES session key,runs the plaintext through Fernet, wraps the session key with ECIES, then returns the ciphertext and wrapped key as a pair.It knows nothing about IPFS, the blockchain, or what the document actually contains it treats the input as raw bytes and produces encrypted output [14], [40].

Module 3 - Upload Module

This module handles the conversation with Pinata.It receives the ciphertext from the Encryption Module, posts it to the Pinata REST endpoint, and passes the returned CID back up the chain.When a revocation triggers an unpin request,this module handles that too.It treats the payload as an opaque binary blob throughout it has no knowledge of what is encrypted inside or who it belongs to [21], [35].

Module 4 - Smart Contract Module

This module covers both the deployed Solidity contract on Ethereum Sepolia and the Web3.py layer Flask uses to reach it.Three functions are exposed to the rest of the system:`shareDocument()` writes a new sharing record to the chain, `accessDocument()` enforces identity and view-count rules before returning a CID, and `revokeAccess()` flips the revocation flag permanently.Every state change goes through an atomic on-chain transaction,is visible on the public ledger, and cannot be undone [30], [42].

Module 5 — Retrieval and Verification Module

This module runs inside Flutter and manages everything on the receiver's side up to the point of decryption. It asks the Smart Contract Module for the list of documents authorised to the current user, calls `accessDocument()` to consume a view and get the CID, pulls the encrypted payload from IPFS using that CID, then recomputes the CID of what actually arrived and checks it against the on-chain value. If the two do not match the file is discarded and a tamper warning is shown. If they match the payload is handed to the Decryption Module [24], [27].

Module 6 — Decryption Module

This module takes the verified payload from Module 5 and turns it back into the original document. It calls Module 1 to unwrap the ECIES-protected session key using the receiver's private key, then feeds that key into Fernet to decrypt the ciphertext. Fernet runs its HMAC check first; a failed check stops everything and surfaces an integrity error. A passing check yields the plaintext byte array, which is sent to the document viewer for rendering.Both the session key and the plaintext are wiped from memory once the viewer has what it needs [27], [40].

Table 5.1: Module Decomposition and Dependencies

Module	**Host**	**Depends On**
1 — Key Management	Flutter App	flutter_secure_storage, eciespy
2 — Encryption	Flask Server	Module 1 (public key), Fernet, eciespy
3 — Upload	Flask Server	Pinata REST API
4 — Smart Contract	Ethereum / Flask	Web3.py, Solidity contract, Sepolia testnet
5 — Retrieval & Verify	Flutter App	Module 4, Pinata IPFS gateway, SHA-256
6 — Decryption	Flutter App	Module 1 (private key), Module 5 (payload)

5.3 Interface Design

The SafeDox interface is organised across four screens that guide users through every stage of the document-sharing lifecycle.Each screen is designed around a single, clear responsibility,keeping the interaction model simple without exposing the complexity of the underlying cryptographic and blockchain operations to the end user.

Login and Registration Screen

The first screen shows how a new user handles identity initialisation. On the first launch, the application creates an Elliptic Curve Cryptography (ECC) key pair which is stored locally on the device. The private key (P_{r_k}) is stored in the hardware-backed secure keychain via `flutter_secure_storage` and never leaves the device.The public key is transmitted to the Flask backend for registration against the user's Ethereum address. Returning users authenticate using their Ethereum address, which is derived deterministically from their stored private key. No username or password is retained on any server;authentication is entirely cryptographic.The screen shows a single input field for the Ethereum address, alongside a *Generate Keys* button for first-time setup,keeping the interface uncluttered while enforcing the zero-trust identity model [27], [42].

Document Upload and Sharing Screen

This screen is the sender's primary workspace.It consists of three interface elements arranged sequentially to reflect the sharing workflow.A file picker button opens the device's native document browser, allowing the sender to select any PDF or image from local storage.Below this,a text field accepts the receiver's Ethereum address as a unique identity token.A numeric input control allows the sender to configure the permitted view count the maximum number of times the recipient may open the document before access is permanently revoked.A single button*Send Securely* ,initiates the complete pipeline which includes: on-device AES-256 encryption, ECC key wrapping,upload to IPFS via Pinata,and smart contract registration on the Ethereum Sepolia testnet,a progress indicator remains visible throughout this pipeline to confirm to the user that the operation is in progress.Upon successful completion,the screen displays the transaction hash and the remaining view count as confirmation [8], [35].

Received Documents Screen

This screen serves as the receiver's document inbox.On loading, the application queries the deployed Solidity smart contract for all document records associated with the receiver's Ethereum address.Each entry in the resulting list displays the sender's address,a truncated document identifier,and the remaining view count.Tapping a document entry triggers the `accessDocument()` smart contract function,which verifies the caller's identity and decrements the view counter atomically.If the check passes,the encrypted file is retrieved from IPFS using the stored Content Identifier (CID).The locally computed CID of the downloaded bytes is compared to the on-chain record before any decryption is attempted.A mismatch causes the application to reject the file and display a tamper-detection warning.On a successful integrity check, the AES session key is unwrapped using the receiver's ECC private key and the document is decrypted entirely on the device before opening in the built-in viewer [24], [27].

Document Viewer Screen

The viewer screen renders the decrypted document within the application.Plaintext data exists only in memory at this stage and is never written to persistent storage on the device.The screen displays the current view count and the maximum permitted views as a clear numeric indicator,so the receiver is aware of how many accesses remain.If the view counter reaches zero on this access,the screen presents a permanent revocation notice on next launch,and all subsequent calls to `accessDocument()` are rejected by the smart contract [42].

Table 5.2: Interface Screen to Architecture Layer Mapping

Screen	**Architecture Layer(s) Involved**	**Primary Operation**
Login / Registration	Layer 1(Flutter), Layer 2(Flask)	ECC key generation; public key registration
Upload /Sharing	Layer 1, Layer 2, Layer 3(IPFS), Layer 4(Blockchain)	AES-256 encryption; IPFS upload; Smart Contract write
Received Documents	Layer 1,Layer 4,Layer 3	Smart contract query; IPFS fetch; CID verification
Document Viewer	Layer 1	On-device AES decryption; in-memory rendering

5.4 Data Structure Design

SafeDox application does not rely on a conventional relational database.Instead,data responsibility is deliberately divided between two complementary stores:the Ethereum smart contract,which holds small,security-critical records on-chain,and IPFS via Pinata,which holds the encrypted file content off-chain.This split follows the established best practice for blockchain-based systems,whereby large binary data is never written directly to the chain; only a compact cryptographic reference is stored on-chain while the payload resides in a distributed storage layer [2], [21].

On-Chain Data Record

Each document shared through SafeDox is represented by a single on-chain record inside the Solidity smart contract.The record is keyed by a documentId an unsigned 256-bit integer incremented monotonically with every call to `shareDocument()`.Table 5.3 describes every field of this structure.

Table 5.3: On-Chain Smart Contract Data Structure (Per Document)

Field Name	Solidity Type	Description
cid	string	IPFS Content Identifier of the encrypted file. Acts as a SHA-256 tamper-detection fingerprint for CID. On an approximate 46-characters(e.g., `QmXn...Kh8`)
senderAddress	address	Ethereum address of the document sender. 20 bytes. Used to restrict `revokeAccess()` to the original owner only.
receiverAddress	address	Ethereum address of the authorised recipient. 20 bytes. The `accessDocument()` function checks `msg.sender` against this field.
viewsPermitted	uint256	The maximum number of times the receiver may open the document. Set once by the sender; never modified after the initial `shareDocument()` call.
viewsRemaining	uint256	Running count of accesses still available. Decremented atomically on each successful call to `accessDocument()`. Reaches zero on the final permitted access.
isRevoked	bool	Revocation flag. Set to `true` by the sender via `revokeAccess()`. Once true, all future `accessDocument()` calls are rejected regardless of `viewsRemaining`.
timestamp	uint256	Unix timestamp recorded by the Ethereum node at the time of sharing. Provides an immutable audit reference for the sharing event.

Three properties make this structure significant from a security standpoint. First,every field is written in a single atomic transaction,so no partial record can exist on-chain.Second,the `viewsRemaining` decrement and the CID retrieval occur within the same `accessDocument()` call,eliminating any race condition where two concurrent requests could consume the same view slot.Third,the "`isRevoked`" flag and the "`viewsRemaining`" counter are evaluated jointly,meaning revocation works independently of whether the view limit has been exhausted [8], [42].

Off-Chain Encrypted Payload

The off-chain payload stored on IPFS consists of two binary components that travel together as a single uploaded object.

Table 5.4: Off-Chain IPFS Payload Structure

Component	Content	Security Property
Fernet ciphertext	AES-256-CBC encrypted bytes of the original document with an appended 'HMAC-SHA256' authentication tag.	Indecipherable without the AES session key. Any modification to the ciphertext causes Fernet to raise an `InvalidSignature` exception during decryption.
ECIES wrapped key	The AES-256 session key encrypted with the receiver's ECC public key using the ECIES.	Recoverable only by the holder of the receiver's ECC private key, which resides exclusively in the device's hardware-backed keychain.

CID returned by IPFS after uploading this payload is a 'SHA-256' hash of the combined binary object.As SHA-256 exhibits the avalanche effect, any single-bit alteration(changes) to either the ciphertext or the wrapped key produces a completely different CID,which immediately fails the on-chain comparison during retrieval [21], [35].

In-Application Data Structures

Within the Flutter mobile application,two lightweight structures manage the user session in memory. Neither is persisted to disk beyond what is necessary for user experience.

UserSession holds the fields required for the current user's identity and key material: `ethereumAddress` (string),`eccPublicKeyPEM` (string), and a boolean `keysInitialised` flag.The ECC private key is never held in this structure;it is accessed on-demand from `flutter_secure_storage` and released from memory after use.

DocumentEntry represents a single record in the received-documents list returned by the smart contract query. It carries "`documentId` (integer)", "`senderAddress` (string)", "`viewsRemaining` (integer)", "`isRevoked` (boolean)" and "`timestamp` (integer)". This structure is populated from the smart contract response on each screen load and is not cached between sessions to ensure that the view count displayed to the user always reflects the current on-chain state [27].

Rationale for Excluding a Central Database

No central database, whether SQLite on the server or a cloud-hosted instance,is required for SafeDox's core security functionality.The smart contract acts as the authoritative,tamper-proof record of every sharing event,permission state,and access history.IPFS serves as the binary storage tier.Introducing a centralised database would create a single point of failure, reintroduce the trust assumptions that the architecture is designed to eliminate, and provide an attack surface that the existing design deliberately avoids [2], [34].An optional local SQLite instance may be used within the Flutter application solely for caching the document list for display purposes,but its contents are always treated as advisory and are never used as a source of truth for access decisions.

5.5 Algorithmic Design

This section describes the step-by-step logic governing SafeDox's four core processes:user initialisation,document encryption and upload,authorised access and decryption,and access revocation.Each algorithm is presented as a numbered procedure followed by a brief complexity and security note.Together these procedures constitute the complete computational specification of the system prior to implementation [27], [40].

Algorithm 1 — User Initialisation and Key Registration

This procedure executes once,the first time a user installs and launches the SafeDox application. It establishes the cryptographic identity that all subsequent operations depend upon(Initilizing step).

Step 1. On first launch:Flutter application checks whether an ECC private key already exists in flutter_secure_storage.

Step 2. If no key is found,the application generates a fresh ECC-256 key pair using the eciespy library on the device.The private key is written to the hardware-backed secure keychain and is never transmitted or logged.The public key is retained in memory for the duration of the registration step.

Step 3. The application generates a new Ethereum address on the Sepolia testnet,derived deterministically from the ECC key material,and stores it in the UserSession structure.

Step 4. The public key(PEM-encoded string) is transmitted to the Flask backend via an 'HTTP POST' request to the /register endpoint,which is paired with the user's Ethereum address as the unique identifier.

Step 5. Flask stores the mapping {ethereumAddress → eccPublicKeyPEM} in its runtime registry.This mapping allows any sender to retrieve a receiver's public key by address before encrypting a document for that recipient.

Step 6. The application sets keysInitialised = true in the UserSession and presents the home screen to the user.

Security note: The private key is generated and stored entirely on-device. Flask receives only the public key, which is mathematically safe to distribute.Even a complete compromise of the Flask server does not expose any user's private key or enable an attacker to decrypt documents [5], [40].

Algorithm 2 — Document Encryption and Upload

This procedure is invoked when the sender taps *Send Securely* after selecting a document and specifying a receiver and view limit.

Step 1. The sender selects a document D from local storage.The file is read into memory as a byte array B_{plain}.

Step 2. Step 2. The Flutter application sends an HTTP GET request to the Flask endpoint /get-public-key/receiverAddress to retrieve the receiver's ECC public key K_{pub}^{recv}.

Step 3. Flask generates a fresh and cryptographically random 256-bit AES session key K_{AES} using Python's secrets module. This key is unique to this document and is never reused again.

Step 4. Flask encrypts B_{plain} using Fernet(AES-256-CBC with HMAC-SHA256) and the session key K_{AES}, to produce ciphertext C and an appended authentication tag τ:

$$C||\tau = \text{Fernet.encrypt}(B_{plain},\ K_{AES}) \tag{i}$$

Step 5. Flask encrypts K_{AES} using the receiver's ECC public key K_{pub}^{recv} through the ECIES scheme, producing a wrapped key W:

$$W = \text{ECIES.encrypt}(K_{AES},\ K_{pub}^{recv}) \tag{ii}$$

The plaintext session key K_{AES} is immediately discarded from memory after this step.

Step 6. Flask uploads the combined payload $\{C||\tau,\ W\}$ to IPFS via the Pinata REST API. Pinata pins the file and returns a Content Identifier *CID*.

Step 7. Flask calls the shareDocument() function on the deployed Solidity smart contract via Web3.py, passing *CID*, the receiver's Ethereum address, and the sender-specified view limit N.The contract writes the record atomically and emits a DocumentShared event on the blockchain.

Step 8. Flask returns the transaction hash and the *CID* to the Flutter application.The app displays a confirmation notice to the sender.

Security note: At no point in this algorithm does the Flask server retain the plaintext document,the raw session key,or the receiver's private key.The server's role is strictly that of a coordinator.If Flask were fully compromised between Steps 4 and 7,an attacker would obtain only ciphertext C and wrapped key W from equations (i) and (ii) — neither of which is decipherable without the receiver's on-device private key [8], [27].

Algorithm 3 — Authorised Access and Decryption

This is used by receiver to gain access and decrypt received document.

Step 1. The receiver opens the *My Documents* screen.The Flutter application calls Flask endpoint /list-documents/{receiverAddress},which queries the smart contract for all `documentId` values whose `receiverAddress` matches the caller.

Step 2. The app populates the document list from the returned DocumentEntry records, displaying senderAddress,viewsRemaining,and `isRevoked` for each entry.

Step 3. The receiver selects a document with `viewsRemaining` > 0 and `isRevoked == false`. The Flutter application calls `accessDocument(documentId)` on the smart contract via web3dart.

Step 4. The smart contract(embedded rules) evaluates the following three conditions individually:

(i) `msg.sender == record.receiverAddress`

(ii) `record.viewsRemaining > 0`

(iii) `record.isRevoked == false`

If all three conditions hold, the contract decrements `viewsRemaining` by one and returns the cid field. If any condition fails, the transaction reverts with an appropriate error message and no state change occurs.

Step 5. The Flutter application fetches the encrypted payload $\{C||\tau,\ W\}$ from IPFS using the returned *CID* via the Pinata gateway.

Step 6. The application recomputes the SHA-256 CID of the downloaded bytes and compares it to the *CID* value returned by the smart contract in Step 4:

$$CID_{computed} \stackrel{?}{=} CID_{on\text{-}chain} \tag{iii}$$

Step 7. The receiver ECC private key K_{priv}^{recv} is retrieved from `flutter_secure_storage`.The wrapped key W is decrypted using ECIES to recover the AES session key K_{AES}:

$$K_{AES} = \text{ECIES.decrypt}(W,\ K_{priv}^{recv}) \tag{iv}$$

Step 8. K_{AES} is used to decrypt the ciphertext C using Fernet. Fernet first verifies the HMAC tag τ;if the tag is invalid,decryption fails.If the tag is valid,the original plaintext byte array B_{plain} is recovered and rendered in the in-app document viewer.

Step 9. K_{AES} and B_{plain} are erased from application memory once rendering is complete. Plaintext is never written to persistent device storage.

Security note: The view-count decrement at Step 4 occurs on-chain before the file is fetched, meaning the consumed view cannot be reclaimed through a network interruption or application crash. The double integrity check at Step 6 —SHA-256 CID comparison shown in equation (iii) — guarantees that any substitution of the file at the IPFS or network layer is detected before decryption is attempted [21], [35].

Algorithm 4 — Access Revocation

This procedure allows the sender to permanently block further access to a document before the view limit is exhausted.

Step 1. The sender locates the target document in the 'Sent Documents' view, which lists all records where `senderAddress` matches the current user.

Step 2. The sender taps *Revoke Access*. The Flutter application calls `revokeAccess(documentId)` on the smart contract via `web3dart`.

Step 3. The smart contract verifies that `msg.sender == record.senderAddress`. If the check fails, the transaction reverts. If it passes, the contract sets `record.isRevoked = true` and emits an `AccessRevoked` event.

Step 4. All subsequent calls to `accessDocument(documentId)` for this record fail condition (iii) of Algorithm 3 Step 4 and are permanently rejected. No further decrement of `viewsRemaining` is possible.

Step 5. Optionally, the sender may call the Pinata API via Flask to unpin the file, removing it from the IPFS network. The on-chain record — including the `cid`, sharing timestamp, and access history — remains on the blockchain permanently as an immutable audit trail.

Security note: Revocation is enforced entirely at the smart contract layer,which means no action by Flask,Pinata or any third party is required to make it effective.Even if `Flask` is offline,the on-chain `isRevoked` flag prevents access. The separation between file availability (IPFS unpinning) and access rights (smart contract flag) ensures that revocation works independently of whether the encrypted file remains physically reachable on IPFS [34], [42].

Chapter 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

SafeDox solves the problem of document safety related to centralized document sharing services like Google Drive and Dropbox by utilizing the decentralized mobile architecture based on IPFS, Ethereum smart contracts, and client-side AES-256 + ECC encryption.The evolution of the state of the art from 2018 to 2025 includes moving from simple Ethereum + IPFS access control to multi-authority ABE, threshold proxy re-encryption,and outsourced ZKP decryption; however, no technique can match the combination of SafeDox's functionality in mobile setting.The core feature that differentiates SafeDox is view-once functionality provided by a Solidity smart contract that decreases view counter and prohibits further access when the limit is exceeded. Encryption occurs solely on the sender side, making the Flask server and IPFS nodes operate with the ciphertext only. IPFS CID provides detection of any file alteration, while Pinata's pinning API avoids the necessity of running IPFS nodes personally.The current limitations of the literature include scalability and Ethereum gas price challenges,access revocation and quantum resistance. The former two are mitigated in SafeDox with the help of Sepolia testnet and simple smart contract design,respectively; quantum resistance is still an open issue.

6.2 Future Work

SafeDox is currently deployed on the Ethereum Sepolia testnet using free test ETH, but future development should focus on deployment on the Ethereum mainnet or cost-effective Layer-2 solutions such as Polygon and Arbitrum to reduce gas fees highlighted in the surveyed literature. Enhancing the system with multi-recipient document sharing, improved Solidity logic for view-based access control,and dynamic revocation models such as time-based expiry, geographic restrictions,and emergency revocation would improve practicality and flexibility.Since AES-256 and ECC may become vulnerable in the era of quantum computing, integrating post-quantum cryptographic algorithms such as CRYSTALS-Kyber and CRYSTALS-Dilithium is an important future direction.

Additional improvements include extending the Flutter application to support iOS,auditing smart contracts using tools like Slither and Mythril,and formally verifying governance logic for enhanced security.The research gap analysis across the reviewed papers also highlights the absence of attribute-based access control and privacy-preserving mechanisms in existing systems therefore, implementing Ciphertext-Policy Attribute-Based Encryption (CP-ABE) and Zero-Knowledge Proofs could provide role-based access control and stronger metadata privacy.Future work may also expand SafeDox into domains such as healthcare and legal document transfer with GDPR and HIPAA compliance support,while replacing reliance on Pinata with fully decentralized IPFS persistence through the Filecoin network.

References

[1] A.-E. Jabri, C. Drocourt, M. Azizi, and G. Utard, "Leveraging Blockchain and Proxy Re-Encryption to Secure Medical IoT Records," in *Proc. Int. Conf.*, 2025.

[2] T. L. Nguyen, L. Nguyen, T. Hoang et al., "Blockchain-Empowered Trustworthy Data Sharing: Fundamentals, Applications, and Challenges," *ACM Comput. Surv.*, 2025.

[3] M. Rangwala, K. R. Venugopal, and R. Buyya, "Blockchain-Enabled Federated Learning," *Book Chapter, Quantum Cloud and Distributed Systems Lab, Univ. of Melbourne*, 2025.

[4] M. R. Haque, S. I. Munna, S. Ahmed, M. T. Islam, M. M. H. Onik, and A. B. M. A. Rahman, "An Integrated Blockchain and IPFS-Based Solution for Secure and Efficient Source Code Repository Hosting Using Middleman Approach," in *Proc. Int. Conf.*, 2025.

[5] A. Ullah, Z. Ullah, S. S. Rizvi, L. Gul, and S. J. Kwon, "Toward Blockchain Based Electronic Health Record Management with Fine-Grained Attribute Based Encryption and Decentralized Storage Mechanisms," *Sci. Rep.*, 2025.

[6] K. R. Kumar, H. Supraja, M. Deekshitha, B. Sireesha, and S. Sadiya, "Block-Chain Based Document Verification System Using IPFS," *Int. J. Creative Res. Thoughts (IJCRT)*, vol. 13, no. 3, 2025.

[7] S. S. N and M. Yadlapalli, "Block Chain-Based Secure Document Sharing," *Indian J. Comput. Sci. Technol.*, vol. 4, no. 1, pp. 204–208, 2025.

[8] N. P. Patil, Y. D. Mane, A. Vasoya, A. Agrawal, and S. Raut, "Secure File Sharing Using Blockchain and IPFS with Smart Contract-Based Access Control," *J. Inf. Syst. Eng. Manage.*, vol. 10, no. 16s, 2025.

[9] M. S. Javed, A. Hennache, M. Imran, and M. K. Khan, "AI-Driven Blockchain and Federated Learning for Secure Electronic Health Records Sharing," *IEEE Access*, 2025.

[10] H. Belfqih and A. Abdellaoui, "Decentralized Blockchain-Based Authentication and Interplanetary File System-Based Data Management Protocol for Internet of Things Using Ascon," *Sensors*, 2025.

[11] J. A. Berrios Moya, J. Ayoade, and M. A. Uddin, "A Zero-Knowledge Proof-Enabled Blockchain-Based Academic Record Verification System," *Electronics*, 2025.

[12] J. Wu, Z. Bian, H. Gao, and Y. Wang, "A Blockchain-Based Secure Data Transaction and Privacy Preservation Scheme in IoT System," *Electronics*, 2025.

[13] M. A. Cardenas-Quispe and A. Pacheco, "Blockchain Ensuring Academic Integrity with a Degree Verification Prototype," *Sci. Rep.*, 2025.

[14] A. Shahzad, W. Chen, Y. Zhang, and R. Kumar, "Zero-Trust Medical Image Sharing: A Secure and Decentralized Approach Using Blockchain and the IPFS," *Electronics*, 2025.

[15] K. M. Sameera, S. Nicolazzo, M. Arazzi, A. Nocera, K. A. Redhiman, V. P, and M. Conti, "Privacy-Preserving in Blockchain-Based Federated Learning Systems," *ACM Comput. Surv.*, 2024.

[16] D. Cai, B. Chen, L. Zhang, K. Li, and H. Kan, "Attribute-Based Encryption With Payable Outsourced Decryption Using Blockchain and Responsive Zero Knowledge Proof," *IEEE Trans. Inf. Forensics Security*, 2024.

[17] R. Tiwari, V. Viswanathan, and S. Rajarajeswari, "Blockchain-Based File Sharing System – A Hybrid Approach," *Int. Res. J. Multidisciplinary Scope (IRJMS)*, vol. 5, no. 3, pp. 1086–1095, 2024.

[18] D. Kadam, T. Dhepe, N. Dhanvi, N. Kamble, and S. Karmode, "Secure File Storage with Blockchain Technology," *Int. J. Progressive Res. Eng. Manage. Sci. (IJPREMS)*, vol. 4, no. 4, pp. 1118–1125, 2024.

[19] J. Agrawal, D. Chaturvedi, J. Jaiswal, and N. Modi, "Web3.0 Document Security: Leveraging Blockchain Technology," *Int. J. Sci. Res. Eng. Manage. (IJSREM)*, vol. 8, no. 4, 2024.

[20] B. Zhang, H. Pan, K. Li, Y. Xing, J. Wang, D. Fan, and W. Zhang, "A Blockchain and Zero Knowledge Proof Based Data Security Transaction Method in Distributed Computing," *Electronics*, 2024.

[21] M. Bin Saif, S. Migliorini, and F. Spoto, "Efficient and Secure Distributed Data Storage and Retrieval Using Interplanetary File System and Blockchain," *Future Internet*, 2024.

[22] S. Yuan, W. Yang, X. Tian, and W. Tang, "A Blockchain-Based Privacy Preserving Intellectual Property Authentication Method," *Symmetry*, vol. 16, no. 5, p. 622, 2024.

[23] E. Goh, D.-Y. Kim, K. Lee, S. Oh, J.-E. Chae, and D.-Y. Kim, "Blockchain-Enabled Federated Learning: A Reference Architecture Design, Implementation, and Verification," *IEEE Access*, 2023.

[24] N. Sangeeta and S. Y. Nam, "Blockchain and Interplanetary File System (IPFS)-Based Data Storage System for Vehicular Networks with Keyword Search Capability," *Electronics*, 2023.

[25] R. Mathwale and R. Ramisetty, "Blockchain Based Inter-Organizational Secure File Sharing System," in *Proc. 2nd Int. Conf. Innovation Technol. (INOCON)*, Bangalore, India, 2023.

[26] R. G. Sonkamble, A. M. Bongale, S. Phansalkar, A. Sharma, and S. Rajput, "Secure Data Transmission of Electronic Health Records Using Blockchain Technology," *Electronics*, 2023.

[27] S. Jadhav, G. Choudhari, and M. Bhavik, "A Decentralized Document Storage Platform Using IPFS with Enhanced Security," in *Proc. 8th Int. Conf. Computing, Commun., Control and Autom. (ICCUBEA)*, Pune, India, 2023.

[28] T. Rahman, S. I. Mouno, A. M. Raatul, A. K. Al Azad, and N. Mansoor, "Verifi-Chain: A Credentials Verifier Using Blockchain and IPFS," in *Proc. Int. Conf.*, 2023.

[29] A. Farabi, I. Khandaker, J. Ahsan, I. K. Shanto, N. Jahan, and M. J. Khan, "ShikkhaChain: A Blockchain-Powered Academic Credential Verification System for Bangladesh," in *Proc. Int. Conf.*, 2023.

[30] M. Hammad, J. Iqbal, C. A. Hassan, S. Hussain, S. S. Ullah, M. Uddin, U. A. Malik, M. Abdelhaq, and R. Alsaqour, "Blockchain-Based Decentralized Architecture for Software Version Control," *Appl. Sci.*, 2023.

[31] M. AlAsqah and T. Moulahi, "Federated Learning and Blockchain Integration for Privacy Protection in the Internet of Things: Challenges and Solutions," *Future Internet*, 2023.

[32] S. A. Sultana, C. Rupa, R. P. Malleswari, and T. R. Gadekallu, "IPFS-Blockchain Smart Contracts Based Conceptual Framework to Reduce Certificate Frauds in the Academic Field," *Information*, 2023.

[33] J. H. Khor, M. Sidorov, and S. A. B. Zulqarnain, "Scalable Lightweight Protocol for Interoperable Public Blockchain-Based Supply Chain Ownership Management," *Sensors*, 2023.

[34] S. Athanere and R. Thakur, "Blockchain Based Hierarchical Semi-Decentralized Approach Using IPFS for Secure and Efficient Data Sharing," *J. King Saud Univ.–Comput. Inf. Sci.*, 2022.

[35] H.-S. Huang, T.-S. Chang, and J.-Y. Wu, "A Secure File Sharing System Based on IPFS and Blockchain," in *Proc. ACM Int. Conf.*, 2022.

[36] S. Anwar, R. Tulsyan, and S. Saha, "AnonChain: A Secure File Sharing Framework Using IPFS Integrated Blockchain," *Int. J. Math., Eng. Manage. Sci.*, vol. 7, no. 6, pp. 844–858, 2022.

[37] U. Tariq, I. Ullah, M. Y. Uddin, and S. J. Kwon, "An Effective Self-Configurable Ransomware Prevention Technique for IoMT," *Sensors*, 2022.

[38] M. Pincheira, E. Donini, M. Vecchio, and S. Kanhere, "A Decentralized Architecture for Trusted Dataset Sharing Using Smart Contracts and Distributed Storage," *Sensors*, 2022.

[39] Y. Zhao, J. Zhao, L. Jiang, R. Tan, D. Niyato, Z. Li, L. Lyu, and Y. Liu, "Privacy-Preserving Blockchain-Based Federated Learning for IoT Devices," *IEEE Internet Things J.*, 2021.

[40] Y. Chen, B. Hu, H. Yu, Z. Duan, and J. Huang, "A Threshold Proxy Re-Encryption Scheme for Secure IoT Data Sharing Based on Blockchain," *Electronics*, 2021.

[41] V. Mani, P. Manickam, Y. Alotaibi, S. Alghamdi, and O. I. Khalaf, "Hyperledger Healthchain: Patient-Centric IPFS-Based Storage of Health Records," *Electronics*, 2021.

[42] M. Steichen, B. Fiz, R. Norvill, W. Shbair, and R. State, "Blockchain-Based, Decentralized Access Control for IPFS," in *Proc. IEEE Int. Conf. Internet of Things, Blockchain, Cybermatics (iThings/GreenCom/CPSCom/SmartData/Blockchain/CIT/Cybermatics)*, 2018.